

NOVA

Documentazione

Gaze-guided Salient Object Ranking

Riccardo Schiavo · Eros Patarini

Industrial Applications of Computer Vision
Università degli Studi di Padova — A.A. 2025-26

26 agosto 2026

Come è organizzato questo documento.

La **Parte I** spiega il funzionamento generale: il problema, la pipeline dall'immagine alle metriche, l'architettura del codice.

La **Parte II** scende nel dettaglio, file per file: per ognuno la funzione che svolge, poi il codice sezionato riga per riga con il motivo di ogni scelta. I file sono presentati in ordine di dipendenza, così non si incontra mai un riferimento a qualcosa non ancora spiegato.

Contents

I	Come funziona il progetto	3
1	Il problema	3
1.1	Input e output	3
2	La pipeline	4
2.1	Un esempio concreto	4
3	L'architettura	4
3.1	I tre livelli	4
3.2	Il contratto della saliency map	5
3.3	La convenzione BGR	5
4	Struttura dei file	5
4.1	Comandi	6
II	Il codice, file per file	7
5	src/baselines/center_prior.py	7
5.1	A cosa serve	7
5.2	La firma	7
5.3	Il corpo, riga per riga	7
5.3.1	Centro e deviazioni standard	7
5.3.2	La griglia di coordinate	8
5.3.3	La Gaussiana	8
5.3.4	La normalizzazione	8
6	src/baselines/spectral_residual.py	9
6.1	L'idea	9
6.2	La pipeline in sette passi	9
6.2.1	Passo 1 — Scala di grigi	9
6.2.2	Passo 2 — Riduzione di risoluzione	10
6.2.3	Passo 3 — FFT e separazione ampiezza/fase	10
6.2.4	Passo 4 — Il residuo	10
6.2.5	Passo 5 — Ricostruzione	10
6.2.6	Passo 6 — Smoothing	11
6.2.7	Passo 7 — Ritorno alla risoluzione originale	11
7	src/baselines/itti_koch.py	11
7.1	L'idea	11
7.2	Le funzioni di supporto	12
7.2.1	_gaussian_pyramid	12
7.2.2	_resize_to	12
7.2.3	_center_surround	13
7.2.4	_normalize_map — l'operatore $N(\cdot)$	13
7.3	La funzione principale	14
7.3.1	Estrazione dei canali	14
7.3.2	Filtri di Gabor	15
7.3.3	Conspicuity map e combinazione finale	15

7.4	Semplificazioni dichiarate	16
8	src/annotation/store.py	16
8.1	Il protocollo	16
8.2	Il formato su disco	16
8.3	<code>__init__</code> — normalizzazione del nome	17
8.4	<code>_save</code> — scrittura atomica	17
8.5	<code>save_annotation</code>	18
8.6	<code>load_all_annotations</code>	18
9	src/annotation/ground_truth.py	18
9.1	Due rappresentazioni, non una	18
9.2	<code>clicks_to_fixation_map</code>	19
9.3	<code>clicks_to_heatmap</code>	19
9.3.1	Il valore di σ	19
9.4	<code>aggregate_annotators_heatmap</code>	19
10	src/evaluation/metrics.py	20
10.1	Le cinque metriche intrinseche	20
10.2	Funzioni di supporto	20
10.3	<code>cc</code> — correlazione di Pearson	21
10.4	<code>sim</code> — histogram intersection	21
10.5	<code>kl_divergence</code>	21
10.6	<code>nss</code>	22
10.7	<code>auc_judd</code>	22
10.7.1	Le soglie	22
10.7.2	La ricerca binaria	22
10.7.3	Il denominatore del false positive rate	22
10.7.4	Un limite matematico da conoscere	23
10.8	Le metriche di ranking	23
10.8.1	<code>rank_data</code> — i ranghi medi	23
10.8.2	<code>spearman</code>	23
10.8.3	<code>kendall_tau</code> — e un errore istruttivo	23
10.8.4	<code>top_k_accuracy</code>	24
11	src/ranking/objects.py	24
11.1	L'architettura del confronto	25
11.2	<code>crop_box</code>	25
11.3	<code>score_box</code> — le tre strategie	25
11.4	<code>rank_by_saliency</code>	26
11.5	<code>rank_by_clicks</code>	26
11.6	<code>aligned_scores</code>	27
12	src/registry.py	27
13	src/visualization.py	28
13.1	<code>saliency_to_heatmap</code>	28
13.2	<code>overlay_saliency</code>	28
13.3	<code>mark_peak</code>	28
14	app.py	28
14.1	Principio architetturale	28
14.2	<code>prepare_image</code>	29
14.3	<code>process</code>	29

14.4	Il selettore a radio button	29
14.5	Collegamento degli eventi	30
15	annotate.py	30
15.1	AnnotationSession	30
15.1.1	start — ripresa automatica	30
15.1.2	save_and_advance	30
15.2	draw_clicks	31
15.3	Il gestore dei click	31
16	scripts/run_evaluation.py	31
16.1	Il controllo di coerenza dimensionale	31
16.2	Gestione dei NaN nelle medie	31
16.3	Le due tabelle prodotte	32
17	tests/test_all.py	32
17.1	Esempi di verità note	32
17.2	Il test che documenta un limite	32
17.3	Isolamento tramite cartelle temporanee	33
18	Riepilogo	33
18.1	Domande di autoverifica	33

Part I

Come funziona il progetto

1 Il problema

La **visual saliency prediction** consiste nel predire quali zone di un'immagine attirano l'attenzione visiva di un osservatore umano. L'output è una *saliency map*: un'immagine in scala di grigi dove il valore di ogni pixel indica quanta attenzione quel punto riceve.

NOVA confronta più modelli di saliency valutandoli contro una ground truth costruita con **click del mouse come proxy del gaze**. Come passo finale, le saliency map vengono combinate con dei bounding box per produrre un **ranking degli oggetti** per importanza visiva.

Perché i click e non un eye-tracker. Un eye-tracker è costoso e non disponibile. La soluzione adottata non è un ripiego improvvisato: è il metodo con cui è stato costruito **SALICON**, uno dei dataset di riferimento della letteratura. Chiedendo a una persona di cliccare sui punti che attirano la sua attenzione si ottiene una distribuzione spaziale fortemente correlata con quella delle fissazioni oculari reali.

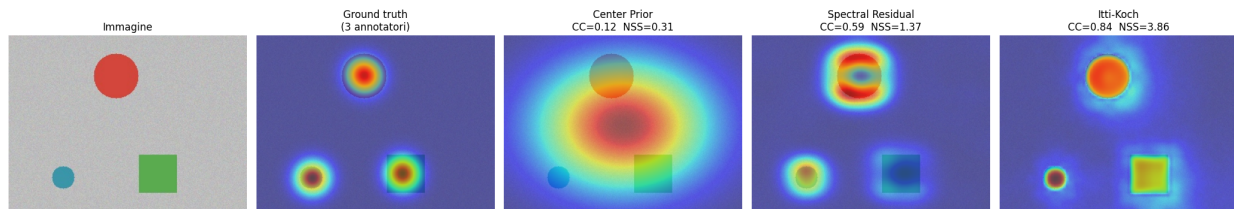
1.1 Input e output

Input	Una singola immagine RGB
Output 1	Una saliency map per ciascun modello: <code>float32</code> , valori in $[0, 1]$, stesse dimensioni dell'immagine
Output 2	Una lista di oggetti ordinati per importanza visiva predetta
Output 3	Tabelle di metriche che quantificano l'accordo di ogni modello con la ground truth

2 La pipeline



2.1 Un esempio concreto



Da sinistra: l'immagine, la ground truth costruita dai click di tre annotatori, e le predizioni dei tre modelli con le rispettive metriche. Si vede il progresso: il center prior illumina il centro ignorando gli oggetti, Spectral Residual li individua parzialmente, Itti-Koch li coglie tutti e tre.

3 L'architettura

3.1 I tre livelli

Livello	Responsabilità
src/baselines/	I modelli: immagine → saliency map
src/registry.py	Uniformare l'accesso ai modelli
src/visualization.py	Saliency map → immagine guardabile
app.py, annotate.py	Solo interfaccia: layout ed eventi

Perché non scrivere tutto in `app.py`. Sarebbe più corto e funzionerebbe identicamente, ma: (1) un'interfaccia grafica non si testa automaticamente, la logica sotto sì; (2) gli script di valutazione riuserebbero le stesse funzioni, che andrebbero duplicate; (3) aggiungere un modello richiederebbe di toccare più punti invece di uno solo.

3.2 Il contratto della saliency map

Tutti i modelli rispettano lo stesso contratto di uscita:

Proprietà	Valore
Tipo	<code>np.float32</code>
Range	<code>[0, 1]</code> — 0 = ignorato, 1 = massima attenzione
Dimensioni	identiche a <code>(height, width)</code> dell'input

Deciso una volta e valido per tutti. Se ogni modello restituisse un formato diverso servirebbero conversioni sparse in tutto il codice, con rischio concreto di confronti falsati.

3.3 La convenzione BGR

OpenCV carica le immagini in ordine **BGR**, non RGB. Tutto il progetto adotta questa convenzione; la conversione a RGB avviene solo al momento di visualizzare.

Errore silenzioso da conoscere. Passare un'immagine RGB a una funzione che si aspetta BGR non solleva nessuna eccezione: la forma è giusta e l'immagine resta visualizzabile. Semplicemente rosso e blu sono scambiati — e Itti-Koch, che ha canali di opponenza cromatica, produce risultati sbagliati senza alcun segnale.

4 Struttura dei file

```
app.py                interfaccia web di confronto
annotate.py          interfaccia web di annotazione
requirements.txt
src/
├── registry.py        registro centrale dei modelli
├── visualization.py   heatmap, overlay, marcatori
├── baselines/
│   ├── center_prior.py    baseline: Gaussiana centrata
│   ├── spectral_residual.py FFT + residuo spettrale
│   └── itti_koch.py       center-surround multi-scala
├── annotation/
│   ├── store.py          persistenza delle annotazioni
│   └── ground_truth.py   click -> heatmap e fissazioni
├── evaluation/
│   └── metrics.py        tutte le metriche
├── ranking/
│   └── objects.py        ranking degli oggetti
├── scripts/run_evaluation.py valutazione -> tabelle CSV
└── tests/test_all.py    56 controlli
```

4.1 Comandi

```
python3 -m venv venv
source venv/bin/activate      # Windows: venv\Scripts\activate
pip install -r requirements.txt

python app.py                 # interfaccia di confronto
python annotate.py --images data/eval  # interfaccia di annotazione
python scripts/run_evaluation.py  # valutazione completa
python tests/test_all.py        # 56 test
```

Part II

Il codice, file per file

I file sono presentati in **ordine di dipendenza**: prima quelli che non dipendono da nessuno, poi quelli che li usano. Leggendoli in quest'ordine non si incontra mai un riferimento a qualcosa non ancora spiegato.

5 src/baselines/center_prior.py

Il modello più semplice del progetto, e il punto migliore da cui iniziare a leggere il codice.

5.1 A cosa serve

Genera una Gaussiana 2D fissa al centro dell'immagine, **ignorando completamente il contenuto**. A parità di dimensioni restituisce sempre la stessa identica mappa.

Perché una baseline così banale è indispensabile. Nella letteratura sull'attenzione visiva è documentato il *center bias*: le persone guardano il centro di una scena molto più spesso della periferia, sia perché chi scatta tende a inquadrare al centro il soggetto, sia per una tendenza oculomotoria naturale.

Un modello che non batte questa Gaussiana non sta estraendo *nessuna* informazione dall'immagine: sta solo riscoprendo il center bias. Le linee guida del corso richiedono esplicitamente almeno una baseline, e questa è la scelta standard per la saliency.

5.2 La firma

```
def center_prior_saliency(image_shape, sigma_frac=0.25):
```

Parametro	Tipo	Significato
image_shape	tuple(int,int)	(altezza, larghezza). Si passa la <i>forma</i> , non l'immagine.
sigma_frac	float	Deviazione standard come frazione delle dimensioni.

Perché la firma accetta la forma e non l'immagine. È una scelta deliberata di *design comunicativo*: la firma stessa dichiara che il modello non guarda mai il contenuto. Accettare l'immagine e poi ignorarla sarebbe funzionalmente identico ma fuorviante per chi legge il codice.

5.3 Il corpo, riga per riga

5.3.1 Centro e deviazioni standard

```
height, width = image_shape

cx = width / 2.0
cy = height / 2.0
sigma_x = sigma_frac * width
sigma_y = sigma_frac * height
```

`cx` e `cy` sono le coordinate del centro geometrico. La divisione è per `2.0` (float) e non per `2` (int) per evitare la divisione intera su immagini di dimensione dispari.

`sigma_x` e `sigma_y` sono **separati e proporzionali** alle rispettive dimensioni. Con un unico σ espresso in pixel:

- su un'immagine 320×240 la macchia coprirebbe una frazione diversa della larghezza rispetto all'altezza — un'ellisse schiacciata rispetto alla forma dell'immagine;
- la stessa scena a risoluzione 4K avrebbe una macchia relativamente minuscola.

5.3.2 La griglia di coordinate

```
x = np.arange(width, dtype=np.float32)
y = np.arange(height, dtype=np.float32)
xx, yy = np.meshgrid(x, y)
```

`np.arange(width)` produce il vettore $[0, 1, 2, \dots, \text{width} - 1]$. `np.meshgrid` li combina in due matrici delle dimensioni dell'immagine:

- `xx[i, j] == j` — la coordinata orizzontale (colonna) di ogni pixel;
- `yy[i, j] == i` — la coordinata verticale (riga) di ogni pixel.

Avendo le coordinate come matrici, la Gaussiana si calcola su tutti i pixel in *una sola* operazione vettorizzata, senza cicli `for` annidati — che in Python sarebbero ordini di grandezza più lenti.

Trappola di indicizzazione. In numpy il primo indice è la *riga* (y), il secondo la *colonna* (x): si scrive `img[y, x]`. È l'inverso della notazione matematica (x, y) . Praticamente tutti gli scambi x/y in questo progetto nascono da qui.

5.3.3 La Gaussiana

```
exponent = -((xx - cx) ** 2 / (2.0 * sigma_x ** 2)
              + (yy - cy) ** 2 / (2.0 * sigma_y ** 2))
gauss = np.exp(exponent)
```

Implementazione diretta della Gaussiana 2D separabile:

$$G(x, y) = \exp\left(-\left[\frac{(x - c_x)^2}{2\sigma_x^2} + \frac{(y - c_y)^2}{2\sigma_y^2}\right]\right)$$

L'esponente è calcolato separatamente per leggibilità. Il valore massimo è 1.0, raggiunto esattamente al centro dove entrambi i termini si annullano.

5.3.4 La normalizzazione

```
def _normalize(saliency_map):
    minimum, maximum = saliency_map.min(), saliency_map.max()
    if maximum - minimum > 0:
        saliency_map = (saliency_map - minimum) / (maximum - minimum)
    else:
        saliency_map = np.zeros_like(saliency_map)
    return saliency_map.astype(np.float32)
```

Normalizzazione *min-max*: sottrae il minimo e divide per l'escursione, portando i valori in $[0, 1]$.

Perché normalizzare se il massimo è già 1.0. Tre motivi: (1) il minimo non è necessariamente 0, quindi il range va comunque riscalo; (2) rende il contratto identico a quello degli altri modelli, dove la normalizzazione è indispensabile; (3) il ramo `else` protegge dal caso degenerare di mappa costante — qui non può accadere, ma negli altri modelli sì, e avere lo stesso schema ovunque riduce il rischio di dimenticarlo dove serve.

Il prefisso `_` nel nome segnala che è una funzione interna al modulo, non parte dell'interfaccia pubblica.

6 src/baselines/spectral_residual.py

Riferimento: Hou & Zhang, *Saliency Detection: A Spectral Residual Approach*, CVPR 2007.

6.1 L'idea

Le immagini naturali, mediamente, hanno uno spettro di ampiezza molto regolare: le basse frequenze dominano e l'energia decade in modo prevedibile al crescere della frequenza. Questa regolarità è la parte **ridondante** di una scena — ciò che tutte le immagini hanno in comune.

L'ipotesi del metodo: *ciò che devia dall'andamento atteso è l'informazione interessante*. Si stima l'andamento atteso sfocando il logaritmo dello spettro, lo si sottrae, e ciò che resta — il **residuo spettrale** — indica le frequenze statisticamente anomale.

Perché funziona pur essendo “solo” un trucco statistico. Un oggetto saliente ha tipicamente bordi netti, una texture diversa dallo sfondo, una forma compatta: tutte caratteristiche che producono energia a frequenze non generiche rispetto al resto della scena. Il metodo non capisce cosa sia un oggetto, ma la statistica è un buon sostituto — e costa circa trenta righe di codice, senza alcun addestramento.

6.2 La pipeline in sette passi

6.2.1 Passo 1 — Scala di grigi

```
original_h, original_w = image.shape[:2]

if image.ndim == 3 and image.shape[2] == 3:
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
else:
    gray = image.copy()
```

Le dimensioni originali si salvano *subito*: serviranno alla fine per riportare la mappa alla risoluzione di partenza.

Il controllo `image.ndim == 3 and image.shape[2] == 3` rende la funzione tollerante: accetta sia immagini a colori sia già in scala di grigi. Il `.copy()` nel ramo `else` evita di lavorare sullo stesso array passato dal chiamante.

Questo passo è la causa del limite più importante del metodo: **l'informazione cromatica viene distrutta**. Un oggetto che si distingue solo per tinta ma non per luminosità diventa invisibile al modello. Il fenomeno è verificato numericamente nella sezione dei test.

6.2.2 Passo 2 — Riduzione di risoluzione

```
small = cv2.resize(gray, target_size,
                    interpolation=cv2.INTER_AREA).astype(np.float32)
```

Il default `target_size=(64, 64)` è il valore del paper originale. La saliency è un fenomeno a bassa frequenza spaziale — interessa *dove* guardare, non i dettagli fini — quindi lavorare a piena risoluzione non aggiunge informazione e costa molto di più.

`INTER_AREA` è l'interpolazione corretta per *rimpicciolire*: calcola la media dei pixel dell'area sorgente che confluisce in ogni pixel destinazione. Le alternative (`INTER_LINEAR`, `INTER_NEAREST`) campionano pochi pixel e producono *aliasing* — artefatti ad alta frequenza che qui sarebbero disastrosi, dato che il metodo analizza proprio lo spettro delle frequenze.

Il cast a `float32` è necessario prima della FFT: su `uint8` i calcoli andrebbero in overflow.

6.2.3 Passo 3 — FFT e separazione ampiezza/fase

```
spectrum = np.fft.fft2(small)
amplitude = np.abs(spectrum)
phase = np.angle(spectrum)
```

`np.fft.fft2` produce una matrice di numeri *complessi*. Ogni numero complesso $z = a + bi$ si può leggere in forma polare come $z = |z|e^{i\theta}$:

- `np.abs(spectrum)` = $|z|$ — l'**ampiezza**: *quanta* energia c'è a quella frequenza;
- `np.angle(spectrum)` = θ — la **fase**: *dove* sono posizionate le strutture nell'immagine.

6.2.4 Passo 4 — Il residuo

```
log_amplitude = np.log(amplitude + 1e-8)
avg_log_amplitude = cv2.blur(log_amplitude, (3, 3))
spectral_residual = log_amplitude - avg_log_amplitude
```

Tre righe, tre decisioni:

`np.log(...)`

L'ampiezza varia di diversi ordini di grandezza tra basse e alte frequenze. Il logaritmo comprime questo range, rendendo l'andamento generale approssimabile con una semplice media locale.

`+ 1e-8`

Protezione da $\log(0)$, che darebbe $-\infty$ e propagherebbe NaN in tutto il calcolo. Il valore è abbastanza piccolo da non alterare i risultati.

`cv2.blur(..., (3,3))`

Filtro a media mobile: stima l'andamento "atteso" dello spettro. La differenza tra spettro reale e atteso è il **residuo**, cioè la parte statisticamente anomala.

6.2.5 Passo 5 — Ricostruzione

```
reconstructed = np.fft.ifft2(np.exp(spectral_residual + 1j * phase))
saliency = (np.abs(reconstructed) ** 2).astype(np.float32)
```

Riga densa, va letta dall'interno verso l'esterno:

1. `spectral_residual + 1j * phase` — ricompone un numero complesso usando il residuo come parte reale (siamo nel dominio logaritmico) e la fase **originale** come parte

immaginaria. `1j` è l'unità immaginaria in Python.

2. `np.exp(...)` — annulla il logaritmo del passo 4, tornando al dominio lineare.
3. `np.fft.ifft2(...)` — FFT inversa: dal dominio delle frequenze allo spazio immagine.
4. `np.abs(...) ** 2` — modulo al quadrato, cioè l'energia. Sempre positiva, come deve essere una saliency map.

Perché si conserva la fase originale. Il metodo modifica solo l'ampiezza. Se si scartasse la fase e se ne usasse una casuale, la ricostruzione non avrebbe alcuna relazione spaziale con la scena: si otterrebbe rumore. La fase è ciò che ancora l'informazione alla posizione.

6.2.6 Passo 6 — Smoothing

```
ksize = int(np.ceil(sigma * 6)) | 1
saliency = cv2.GaussianBlur(saliency, (ksize, ksize), sigma)
```

`cv2.GaussianBlur` richiede una dimensione di kernel *dispari*.

`sigma * 6`

Regola pratica: una Gaussiana è praticamente nulla oltre $\pm 3\sigma$, quindi un kernel di 6σ la contiene quasi interamente.

`| 1`

OR bit-a-bit con 1: forza a 1 il bit meno significativo, trasformando qualsiasi numero pari nel dispari successivo ($12 \rightarrow 13$, $13 \rightarrow 13$). Idioma compatto per “arrotonda al dispari”.

Lo smoothing non è cosmetico: la ricostruzione grezza è molto puntinata, mentre l'attenzione umana si concentra su *regioni*, non su singoli pixel isolati.

6.2.7 Passo 7 — Ritorno alla risoluzione originale

```
saliency = _normalize(saliency)
return cv2.resize(saliency, (original_w, original_h),
                  interpolation=cv2.INTER_LINEAR)
```

Qui si usa `INTER_LINEAR` perché si sta *ingrandendo*, il caso opposto al passo 2.

`cv2.resize` vuole la dimensione come `(width, height)`, mentre `shape` di numpy è `(height, width)`. L'ordine invertito è una fonte classica di bug silenziosi: l'immagine esce distorta senza alcun errore.

7 src/baselines/itti_koch.py

Riferimento: Itti, Koch & Niebur, *A Model of Saliency-Based Visual Attention for Rapid Scene Analysis*, IEEE TPAMI 1998. Il file più articolato del progetto: una funzione pubblica e quattro funzioni di supporto.

7.1 L'idea

Un punto è saliente se spicca rispetto a ciò che lo circonda. Il modello non riconosce oggetti: misura contrasto locale su tre canali che il sistema visivo umano elabora in parallelo nelle prime fasi della visione.

Canale	Corrispettivo biologico
Intensità	Contrasto di luminanza, la via più antica ed elementare.
Colore	Opponenza rosso/verde e blu/giallo: è il modo in cui la retina codifica effettivamente il colore, tramite cellule gangliari antagoniste.
Orientazione	Filtri di Gabor a 0°, 45°, 90°, 135°: modello classico delle cellule semplici della corteccia visiva primaria (V1).

7.2 Le funzioni di supporto

7.2.1 `_gaussian_pyramid`

```
def _gaussian_pyramid(channel, levels):
    pyramid = [channel.astype(np.float32)]
    for _ in range(levels - 1):
        pyramid.append(cv2.pyrDown(pyramid[-1]).astype(np.float32))
    return pyramid
```

Restituisce una *lista* di array: `pyramid[0]` è l'immagine originale, `pyramid[1]` è a metà risoluzione, `pyramid[2]` a un quarto, e così via.

Ogni livello si costruisce da `pyramid[-1]`, cioè dall'ultimo aggiunto: la riduzione è cumulativa.

A cosa serve la piramide. Permette di guardare la scena “a più distanze”: un dettaglio fine e un oggetto grande producono contrasto a scale diverse, e senza la piramide se ne coglierebbe solo una.

`cv2.pyrDown` non si limita a scartare pixel: applica prima una sfocatura Gaussiana e poi sottocampiona. Senza la sfocatura si avrebbe aliasing.

7.2.2 `_resize_to`

```
def _resize_to(feature_map, shape):
    return cv2.resize(feature_map, (shape[1], shape[0]),
                      interpolation=cv2.INTER_LINEAR)
```

Wrapper minimale che esiste per un solo motivo: **isolare in un punto unico** l'inversione `(height, width) → (width, height)`. Lo scambio `shape[1], shape[0]` compare così una volta sola nel modulo, invece che in ogni punto in cui serve un `resize`.

7.2.3 _center_surround

```
def _center_surround(pyramid, center_scales=(2, 3), deltas=(3, 4)):
    maps = []
    for c in center_scales:
        for delta in deltas:
            s = c + delta
            if s >= len(pyramid):
                continue
            center = pyramid[c]
            surround = _resize_to(pyramid[s], center.shape[:2])
            maps.append(np.abs(center - surround))
    return maps
```

Doppio ciclo su $c \in \{2, 3\}$ (scala *center*, il dettaglio) e $\delta \in \{3, 4\}$, con $s = c + \delta$ (scala *surround*, il contesto). Le quattro combinazioni sono (2, 5), (2, 6), (3, 6), (3, 7).

Il `continue` salta le combinazioni in cui s eccede i livelli disponibili: con `pyramid_levels=7` (indici 0–6) la coppia (3, 7) viene scartata, restano tre mappe per canale.

Prima della sottrazione, la scala grossolana viene riportata alla risoluzione di quella fine: sono array di dimensioni diverse e non si potrebbero sottrarre direttamente. Il valore assoluto serve perché conta l'*entità* del contrasto, non il suo segno — chiaro su scuro e scuro su chiaro sono entrambi salienti.

È l'analogo computazionale dei campi recettivi center-surround dei neuroni retinici.

7.2.4 _normalize_map — l'operatore $N(\cdot)$

La funzione concettualmente più densa del progetto.

```
def _normalize_map(feature_map, M=10.0):
    feature_map = feature_map - feature_map.min()
    if feature_map.max() > 0:
        feature_map = feature_map / feature_map.max() * M

    local_max = cv2.dilate(feature_map, np.ones((7, 7), np.uint8))
    is_local_max = (feature_map == local_max) & (feature_map < M - 1e-3)
    peaks = feature_map[is_local_max]
    mean_peak = peaks.mean() if peaks.size > 0 else 0.0

    return feature_map * (M - mean_peak) ** 2
```

Righe 1-3: portare la mappa in $[0, M]$. Scala lineare in un intervallo noto, con $M = 10$ (valore del paper). La scelta esatta è arbitraria: conta il *rapporto* fra M e la media dei picchi.

Riga 5: trovare i massimi locali con una dilatazione. `cv2.dilate` sostituisce ogni pixel con il massimo della sua vicinanza 7×7 . Di conseguenza, un pixel che dopo la dilatazione *vale ancora quanto prima* è esattamente un massimo locale. È un trucco elegante: identifica tutti i massimi locali con una singola operazione morfologica invece che con una scansione esplicita.

Riga 6: escludere il massimo globale. La condizione `feature_map < M - 1e-3` scarta il picco più alto, che vale esattamente M dopo la normalizzazione. Interessa la media degli *altri* picchi: è quella che dice se la mappa ha un vincitore netto o tanti competitori.

Righe 7-8: la media dei picchi, con guardia. `peaks.size > 0` protegge dal caso in cui non ci siano massimi locali oltre al globale: `.mean()` su un array vuoto restituirebbe NaN con un warning.

Riga 10: il fattore di enfasi.

$$N(\text{mappa}) = \text{mappa} \times (M - \bar{m})^2$$

dove $\bar{m}$ è la media dei massimi locali.

Situazione	$\bar{m}$	Effetto
Un solo picco netto	bassa	$(M - \bar{m})^2$ grande → mappa amplificata
Molti picchi simili	alta	$(M - \bar{m})^2$ piccolo → mappa soppressa

Perché conta. Due mappe possono avere lo stesso valore massimo ma significato opposto: una con un picco isolato indica un elemento davvero distintivo, una con venti picchi di altezza simile indica rumore diffuso.

Senza $N(\cdot)$ un canale rumoroso dominerebbe la somma finale per pura forza bruta. È il meccanismo che rende il modello *selettivo* invece che additivo — ed è anche la parte del paper originale che viene più spesso semplificata nelle reimplementazioni, inclusa la nostra.

7.3 La funzione principale

7.3.1 Estrazione dei canali

```
img = image.astype(np.float32)
b, g, r = cv2.split(img)

intensity = (r + g + b) / 3.0

eps = 1e-3
rg = (r - g) / (intensity + eps)
by = (b - np.minimum(r, g)) / (intensity + eps)
```

`cv2.split` separa i tre canali. L'ordine di destrutturazione è `b, g, r` perché OpenCV usa BGR.

Le formule di opponenza cromatica riproducono il modo in cui la retina umana codifica il colore: non come tre canali indipendenti, ma come *differenze* tra coppie antagoniste. Il giallo è approssimato da `np.minimum(r, g)`, dato che percettivamente è la componente comune a rosso e verde.

Perché dividere per l'intensità. Normalizza rispetto alla luminosità: un rosso acceso in ombra e lo stesso rosso in piena luce hanno valori RGB molto diversi ma la stessa *tinta*. Dividendo per `intensity` il canale misura la tinta indipendentemente dall'illuminazione.

`eps = 1e-3` evita la divisione per zero nelle zone quasi nere, dove il colore è comunque privo di significato percettivo.

7.3.2 Filtri di Gabor

```
orientation_pyramids = []
for theta_deg in (0, 45, 90, 135):
    kernel = cv2.getGaborKernel(
        ksize=(15, 15), sigma=4.0, theta=np.radians(theta_deg),
        lambd=10.0, gamma=0.5, psi=0,
    )
    orientation_pyramids.append(
        [cv2.filter2D(level, cv2.CV_32F, kernel) for level in intensity_pyr]
    )
```

Un filtro di Gabor è una senoide modulata da una Gaussiana: risponde fortemente ai bordi che hanno l'orientamento e la frequenza spaziale a cui è sintonizzato.

Parametro	Valore	Significato
ksize	(15,15)	Dimensione del kernel: deve contenere almeno un periodo della senoide.
sigma	4.0	Ampiezza dell'involuppo Gaussiano: quanto è esteso il filtro.
theta	0-135°	Orientamento a cui è sensibile. <code>np.radians</code> converte: OpenCV vuole radianti.
lambd	10.0	Lunghezza d'onda della senoide: la "spaziatura" dei bordi cercati.
gamma	0.5	Rapporto d'aspetto: allunga il filtro, rendendolo più selettivo sull'orientamento.
psi	0	Sfasamento. A 0 il filtro è simmetrico.

Il filtro si applica a *tutti* i livelli della piramide di intensità: da qui la list comprehension interna. Il risultato sono 4 piramidi, una per orientamento.

`cv2.CV_32F` forza l'output in float: la risposta di Gabor può essere negativa e su `uint8` verrebbe troncata.

7.3.3 Conspicuity map e combinazione finale

```
target_shape = intensity_maps[0].shape

def combine(maps):
    resized = [_resize_to(m, target_shape) for m in maps]
    return _normalize_map(np.sum(resized, axis=0))

conspicuity_intensity = combine(intensity_maps)
conspicuity_color = combine(color_maps)
conspicuity_orientation = combine(orientation_maps)

saliency = (conspicuity_intensity
            + conspicuity_color
            + conspicuity_orientation) / 3.0
```

Le mappe center-surround provengono da scale diverse e hanno quindi *dimensioni diverse*: `combine` le riporta tutte a una risoluzione comune prima di sommarle. `np.sum(resized, axis=0)`

somma lungo l'asse della lista, cioè elemento per elemento.

Notare la **doppia normalizzazione**: prima su ogni singola mappa center-surround, poi di nuovo sulla somma. Serve perché il confronto avviene a due livelli — prima tra le scale dello stesso canale, poi tra i tre canali.

La media finale è *non pesata*: i pesi ottimali sarebbero un parametro da apprendere, e il modello originale è deliberatamente privo di apprendimento.

7.4 Semplificazioni dichiarate

Rispetto al paper originale:

1. Coppie di scale center-surround fisse ($c \in \{2, 3\}$, $\delta \in \{3, 4\}$) invece di un set più ampio.
2. Operatore $N(\cdot)$ a singolo passaggio invece che iterativo.

Entrambe riducono il costo computazionale mantenendo il comportamento qualitativo. **Vanno dichiarate nel README**: una semplificazione documentata è una scelta di progetto, una non documentata è un errore.

8 src/annotation/store.py

Gestisce la lettura e la scrittura delle annotazioni.

8.1 Il protocollo

L'annotatore guarda un'immagine e clicca **in ordine di importanza decrescente**. Minimo 3 click, massimo 6.

```
MIN_CLICKS = 3 # sotto, non esiste un ranking sensato
MAX_CLICKS = 6 # sopra, si cercano cose da cliccare invece di notarle
```

Un compito, due ground truth. L'ordine dei click non è un dettaglio accessorio: rende lo stesso gesto utile per due scopi.

- i click come *punti di fissazione* → metriche NSS, AUC-Judd;
- l'ordine come *ranking* di importanza → Spearman, Kendall, Top-1.

Un secondo giro di annotazione per il ranking raddoppierebbe il lavoro degli annotatori, che è la risorsa più scarsa.

8.2 Il formato su disco

Un file JSON **per annotatore**, non un file unico:

```
data/annotations/riccardo.json

{
  "annotator": "riccardo",
  "created_at": "2026-08-26T18:00:00+00:00",
  "annotations": {
    "img_001.jpg": {
      "image_size": [480, 640],
      "clicks": [{"x": 412, "y": 230, "order": 1}, ...]
    }
  }
}
```

Perché un file per annotatore. Più persone possono annotare in parallelo senza conflitti di scrittura, ed escludere un annotatore a posteriori diventa banale (si cancella il suo file). Un file unico richiederebbe un meccanismo di lock, o produrrebbe sovrascritture silenziose.

Perché salvare `image_size`. Senza le dimensioni, le coordinate sono prive di significato: se l'immagine venisse ridimensionata o sostituita, i click punterebbero altrove senza che nulla lo segnali.

8.3 `__init__` — normalizzazione del nome

```
if not annotator or not annotator.strip():
    raise ValueError("Il nome dell'annotatore non puo' essere vuoto.")

self.annotator = annotator.strip().lower().replace(" ", "_")
self.annotations_dir = annotations_dir
self.path = os.path.join(annotations_dir, f"{self.annotator}.json")
self._data = self._load()
```

La catena `strip().lower().replace(" ", "_")` evita che “Riccardo”, “riccardo” e “Riccardo” diventino tre annotatori distinti — errore facile quando si digita il nome a inizio sessione.

8.4 `_save` — scrittura atomica

```
def _save(self):
    os.makedirs(self.annotations_dir, exist_ok=True)
    tmp_path = self.path + ".tmp"
    with open(tmp_path, "w", encoding="utf-8") as f:
        json.dump(self._data, f, indent=2, ensure_ascii=False)
    os.replace(tmp_path, self.path)
```

Si scrive su un file temporaneo e poi lo si rinomina.

Perché non scrivere direttamente sul file finale. Se il processo venisse interrotto a metà scrittura (crash, chiusura del terminale), il file resterebbe troncato — JSON non valido, *tutte* le annotazioni precedenti perse.

`os.replace` è **atomico** sulla maggior parte dei filesystem: o il file nuovo sostituisce il vecchio per intero, o non succede nulla. Con sessioni da decine di minuti, questa protezione non è teorica.

`ensure_ascii=False` preserva gli accenti nei nomi invece di trasformarli in sequenze di escape.

8.5 save_annotation

```
if not MIN_CLICKS <= len(clicks) <= MAX_CLICKS:
    raise ValueError(
        f"Servono da {MIN_CLICKS} a {MAX_CLICKS} click, ricevuti {len(clicks)}."
    )
self._data["annotations"][image_name] = {
    "image_size": list(image_size),
    "clicks": [
        {"x": int(x), "y": int(y), "order": i + 1}
        for i, (x, y) in enumerate(clicks)
    ],
    "annotated_at": datetime.now(timezone.utc).isoformat(),
}
self._save()
```

L' `order` si deriva dalla *posizione nella lista*: `enumerate(clicks)` con `i + 1` produce 1, 2, 3... Il chiamante non deve tracciarlo, quindi non può sbagliarlo.

Il salvataggio avviene a ogni chiamata, non a fine sessione: è il salvataggio incrementale.

8.6 load_all_annotations

```
for filename in sorted(os.listdir(annotations_dir)):
    if not filename.endswith(".json"):
        continue
    ...
    for image_name, record in data.get("annotations", {}).items():
        if image_name not in aggregated:
            aggregated[image_name] = {
                "image_size": record["image_size"],
                "annotators": {},
            }
        aggregated[image_name]["annotators"][annotator] = record["clicks"]
```

Inverte l'organizzazione dei dati: da *per annotatore* a *per immagine*. È la forma utile per costruire la ground truth, che deve unire le opinioni di tutti su una stessa immagine.

Il filtro `.endswith(".json")` ignora i file `.tmp` eventualmente rimasti da una scrittura interrotta.

9 src/annotation/ground_truth.py

Converte i click in ground truth utilizzabile dalle metriche.

9.1 Due rappresentazioni, non una

Rappresentazione	Forma	Metriche che la usano
Mappa di fissazione	binaria, sparsa	NSS, AUC-Judd
Heatmap continua	Gaussiane sommate	CC, SIM, KL

Non è ridondanza: NSS chiede “quanto vale la saliency *esattamente* sui punti guardati?” e sfumare i punti falserebbe la domanda; CC confronta due distribuzioni, e la ground truth deve essere continua quanto la predizione.

È la stessa doppia rappresentazione dei benchmark di riferimento (MIT/Tübingen).

9.2 clicks_to_fixation_map

```
height, width = image_size
fixation_map = np.zeros((height, width), dtype=np.float32)

for click in clicks:
    x = int(np.clip(click["x"], 0, width - 1))
    y = int(np.clip(click["y"], 0, height - 1))
    fixation_map[y, x] = 1.0

return fixation_map
```

Il `np.clip` protegge da coordinate fuori dai bordi: un click sul bordo estremo dell'interfaccia può produrre un valore pari alla larghezza, che come indice sarebbe fuori range. Senza clip si avrebbe un `IndexError` a metà sessione, con perdita del lavoro in corso.

Notare `fixation_map[y, x]` — riga prima, colonna dopo.

9.3 clicks_to_heatmap

```
sigma = sigma_frac * width
y_grid, x_grid = np.mgrid[0:height, 0:width].astype(np.float32)

for click in clicks:
    squared_distance = (x_grid - click["x"]) ** 2 + (y_grid - click["y"]) ** 2
    heatmap += np.exp(-squared_distance / (2.0 * sigma ** 2))

max_value = heatmap.max()
if max_value > 0:
    heatmap /= max_value
```

`np.mgrid[0:height, 0:width]` costruisce le griglie di coordinate in una riga, ed è calcolata *fuori* dal ciclo: viene riusata per ogni click invece di essere ricostruita.

Le Gaussiani si **sommano**. Dove più click sono vicini i contributi si sovrappongono e il valore cresce: è così che il consenso tra annotatori emerge automaticamente.

9.3.1 Il valore di σ

```
DEFAULT_SIGMA_FRAC = 0.05
```

Non è un numero arbitrario. Il 5% della larghezza approssima l'ampiezza della fovea — circa 1 grado di angolo visivo — alle distanze di osservazione tipiche di uno schermo.

L'idea sottostante: quando si guarda un punto non si vede nitidamente *solo* quel pixel. La visione ad alta risoluzione copre una piccola regione attorno al punto di fissazione, e la Gaussiana modella esattamente questo.

È espresso come frazione e non in pixel, per lo stesso motivo del center prior: coerenza a risoluzioni diverse. È una scelta da saper giustificare all'orale.

9.4 aggregate_annotators_heatmap

```
all_clicks = []
for clicks in annotators_clicks.values():
    all_clicks.extend(clicks)
return clicks_to_heatmap(all_clicks, image_size, sigma_frac=sigma_frac)
```

Tutti i click di tutti gli annotatori confluiscono in **un'unica nuvola di punti**, da cui si genera una sola heatmap.

L'alternativa scartata. Costruire una heatmap per annotatore e poi farne la media sembra equivalente, ma non lo è: la normalizzazione individuale (dividere per il massimo) amplificherebbe il contributo di chi ha fatto 3 click rispetto a chi ne ha fatti 6. Aggregando prima e normalizzando dopo, ogni click vale uguale.

10 src/evaluation/metrics.py

Tutte le metriche del progetto, implementate da zero.

Perché non usare `scipy.stats`. Sono poche righe, e le convenzioni — gestione dei pari merito, ordine degli argomenti di KL, denominatore del false positive rate — sono esattamente ciò che va capito e saputo spiegare. Implementarle è parte del progetto, non una perdita di tempo.

10.1 Le cinque metriche intrinseche

Metrica	Ground truth	Intervallo	Direzione
CC	heatmap continua	$[-1, 1]$	alto = meglio
SIM	heatmap continua	$[0, 1]$	alto = meglio
KL	heatmap continua	$[0, \infty)$	basso = meglio
NSS	fissazioni discrete	$(-\infty, \infty)$	alto = meglio
AUC-Judd	fissazioni discrete	$[0, 1]$	alto = meglio

KL va nella direzione opposta rispetto a tutte le altre. Nelle tabelle di confronto va segnalato esplicitamente, altrimenti si legge il risultato al contrario.

Perché cinque metriche e non una. Ognuna è cieca a qualcosa. CC ignora dove sta il picco purché l'andamento generale corrisponda; NSS guarda solo i punti fissati e ignora tutto il resto; AUC-Judd è invariante a qualsiasi trasformazione monotona, quindi conta l'ordinamento ma non l'intensità dei picchi.

Un modello che produce una mappa perfetta ma spostata di dieci pixel avrà CC ancora alto e NSS crollato. Riportare una sola metrica significa nascondere metà del comportamento.

10.2 Funzioni di supporto

```
def _to_distribution(saliency_map):
    m = saliency_map.astype(np.float64)
    m = m - m.min()
    total = m.sum()
    if total < EPS:
        return np.full_like(m, 1.0 / m.size)
    return m / total
```

Riscalda la mappa perché la somma dei valori sia 1, rendendola una distribuzione di probabilità. Serve a SIM e KL.

Il cast a `float64` non è pedanteria: KL somma logaritmi su decine di migliaia di pixel,

e in `float32` l'errore di arrotondamento si accumula in modo visibile.

Il caso degenere (mappa costante) restituisce la distribuzione uniforme, l'unica sensata.

```
def _standardize(saliency_map):
    m = saliency_map.astype(np.float64)
    std = m.std()
    if std < EPS:
        return np.zeros_like(m)
    return (m - m.mean()) / std
```

Z-score: media 0, deviazione standard 1. Rende confrontabili mappe prodotte su scale diverse ed è il cuore di CC e NSS.

10.3 cc — correlazione di Pearson

```
def cc(saliency_map, ground_truth_map):
    _check_shape(saliency_map, ground_truth_map)
    return float((_standardize(saliency_map) * _standardize(ground_truth_map)).mean())
```

Perché una riga basta. La correlazione di Pearson è $\text{cov}(S, G) / (\sigma_S \sigma_G)$. Standardizzando entrambe le mappe, le deviazioni standard diventano 1 e le medie 0: la formula si riduce alla media del prodotto. Non serve implementare covarianza e deviazioni separatamente.

Conseguenza dell'invarianza: una mappa e la stessa mappa moltiplicata per 10 hanno $CC = 1$. Il modello non deve essere calibrato, solo corretto nell'andamento.

10.4 sim — histogram intersection

```
s = _to_distribution(saliency_map)
g = _to_distribution(ground_truth_map)
return float(np.minimum(s, g).sum())
```

Entrambe le mappe diventano distribuzioni a somma 1; si somma il minimo punto per punto: quanta massa di probabilità è condivisa. Con distribuzioni identiche il minimo coincide con la mappa stessa, e la somma vale 1.

10.5 kl_divergence

```
support = g > 0
if not support.any():
    return 0.0
return float(np.sum(g[support] * np.log(g[support] / np.maximum(s[support], EPS))))
```

Tre dettagli, tutti emersi in fase di test:

La maschera `g > 0`

Dove la ground truth è nulla, il termine $g \log(g/s)$ vale 0 per definizione (limite di $x \log x$ per $x \rightarrow 0$). Escludere quei punti è matematicamente corretto e più rapido.

`np.maximum(s, EPS)` e non `s + EPS`

Differenza sostanziale. Sommare `EPS` altera il rapporto *ovunque*, introducendo un errore di $-\varepsilon$ per pixel che si accumula: su una mappa da 4800 pixel, KL di una distribuzione con se stessa dava -4.7×10^{-9} invece di 0. Con `maximum`, `s` viene usato esattamente dove è significativo, ed `EPS` interviene solo dove `s` è nullo.

L'ordine degli argomenti

KL è **asimmetrica**: $KL(a||b) \neq KL(b||a)$. La convenzione della letteratura è $KL(\text{ground truth} || \text{predizione})$, che punisce severamente il modello che assegna saliency quasi nulla dove la ground truth ne ha molta — nell'uso reale mancare un oggetto importante è più grave che segnalarne uno di troppo.

10.6 nss

```
fixations = fixation_map > 0
if not fixations.any():
    return float("nan")
return float(_standardize(saliency_map)[fixations].mean())
```

Standardizza la mappa predetta e ne prende il valore medio *esattamente sui punti fissati*. L'indicizzazione booleana `[fixations]` estrae solo quei valori.

Il valore 0 corrisponde alla prestazione casuale: le fissazioni cadono su punti di saliency media. Valori positivi indicano che cadono su zone più salienti della media.

Senza fissazioni si restituisce `nan` invece di 0: 0 significherebbe “prestazione casuale” e falserebbe le medie sul dataset.

10.7 auc_judd

La più elaborata. Interpreta la saliency map come un classificatore binario: al variare della soglia, quante fissazioni vengono catturate rispetto a quanta immagine viene selezionata?

```
fixation_values = np.sort(s[fixations])[:-1]
all_values = np.sort(s.ravel())
n_fixations = fixation_values.size
n_pixels = all_values.size
n_non_fixations = n_pixels - n_fixations

for i, threshold in enumerate(fixation_values):
    above = n_pixels - np.searchsorted(all_values, threshold, side="left")
    tp_rates.append((i + 1) / n_fixations)
    fp_rates.append((above - (i + 1)) / n_non_fixations)
```

10.7.1 Le soglie

Si usano i valori di saliency *nei punti fissati*, ordinati in modo decrescente. Sono gli unici punti in cui il true positive rate cambia: fra due valori consecutivi il numeratore resta costante, quindi bastano a descrivere tutta la curva.

10.7.2 La ricerca binaria

`np.searchsorted` su un array già ordinato costa $O(\log n)$. L'alternativa ingenua `(s >= threshold).sum()` costa $O(n)$ e va ripetuta per ogni fissazione: su un'immagine 800×600 con 18 fissazioni la differenza è già misurabile.

10.7.3 Il denominatore del false positive rate

`(above - (i+1)) / n_non_fixations`, non `above / n_pixels`. Le fissazioni sopra soglia sono veri positivi: contarle anche come falsi positivi gonfierebbe l'asse orizzontale della curva ROC. Si sottraggono dal numeratore e si escludono dal denominatore.

10.7.4 Un limite matematico da conoscere

Con una **sola** fissazione la curva ROC ha soltanto tre punti — $(0,0)$, $(fp,1)$, $(1,1)$ — e la sua area vale $1 - fp/2$. Il minimo è quindi **0.5**, raggiunto quando $fp = 1$ (fissazione sul valore minimo della mappa).

Servono almeno due fissazioni perché l'AUC possa scendere sotto 0.5 e segnalare una predizione peggiore del caso. Questo comportamento è emerso scrivendo i test: l'aspettativa iniziale ("fissazione sul minimo → AUC vicina a 0") era matematicamente impossibile. Il test è stato corretto, non il codice — ma solo dopo aver verificato su carta quale dei due sbagliasse.

10.8 Le metriche di ranking

10.8.1 rank_data — i ranghi medi

```
order = np.argsort(values, kind="mergesort")
sorted_values = values[order]

i = 0
while i < n:
    j = i
    while j + 1 < n and sorted_values[j + 1] == sorted_values[i]:
        j += 1
    ranks[order[i:j + 1]] = (i + j) / 2.0 + 1.0
    i = j + 1
```

Converte valori in ranghi assegnando la **media** in caso di pari merito: $[10, 20, 20, 40] \rightarrow [1, 2.5, 2.5, 4]$. I due valori uguali occuperebbero le posizioni 2 e 3: entrambi ricevono 2.5.

Il ciclo interno individua l'estensione di ogni gruppo di valori identici; `order[i:j+1]` riporta il rango medio alle posizioni originali.

`kind="mergesort"` garantisce un ordinamento *stabile*: a parità di valore l'ordine originale è preservato, rendendo il risultato deterministico.

10.8.2 spearman

```
rp = rank_data(predicted)
rt = rank_data(reference)
rp = rp - rp.mean()
rt = rt - rt.mean()

denominator = np.sqrt((rp ** 2).sum() * (rt ** 2).sum())
if denominator < EPS:
    return float("nan")
return float((rp * rt).sum() / denominator)
```

È la correlazione di Pearson calcolata sui *ranghi* invece che sui valori. Cattura qualsiasi relazione monotona: $[1, 2, 3, 4]$ e $[10, 200, 3000, 40000]$ hanno $\rho = 1$ pur non essendo linearmente correlati.

Si usa la formula generale e non la scorciatoia $1 - 6 \sum d^2 / n(n^2 - 1)$, perché quest'ultima vale solo in assenza di pari merito.

10.8.3 kendall_tau — e un errore istruttivo

$$\tau_b = \frac{C - D}{\sqrt{(n_0 - n_1)(n_0 - n_2)}}$$

dove n_0 è il numero totale di coppie, n_1 le coppie in pari merito nella predizione, n_2 quelle in pari merito nella ground truth.

```
for i in range(n - 1):
    for j in range(i + 1, n):
        dp = predicted[i] - predicted[j]
        dt = reference[i] - reference[j]

        if dp == 0:
            ties_predicted += 1
        if dt == 0:
            ties_reference += 1

        if dp != 0 and dt != 0:
            if dp * dt > 0:
                concordant += 1
            else:
                discordant += 1

n_pairs = n * (n - 1) // 2
denominator = np.sqrt((n_pairs - ties_predicted) * (n_pairs - ties_reference))
```

L'errore scoperto dai test. La prima versione usava $\sqrt{(C + D + n_1)(C + D + n_2)}$, che sembra equivalente ma non lo è.

$n_0 - n_1$ conta le coppie *non* in pari merito nella predizione, mentre $C + D + n_1$ conta quelle decise più quelle pari. Le due espressioni coincidono quando nessuna coppia è in pari merito in **entrambe** le sequenze — e anche quando differiscono, il *prodotto* dei due fattori resta identico perché i termini si scambiano.

Risultato: l'errore era invisibile in tutti i test casuali e anche nel confronto con `scipy`. È emerso solo sul caso limite `kendall_tau([5,5,5], [5,5,5])`, dove la formula corretta dà $\sqrt{0 \cdot 0} = 0$ e quindi `nan` (“metrica non definita”), mentre quella sbagliata dava $0/3 = 0$ (“nessuna correlazione”). Significati completamente diversi.

10.8.4 top_k_accuracy

```
true_best = int(np.argmax(reference))
top_k_predicted = np.argsort(predicted)[::-1][:k]
return float(true_best in top_k_predicted)
```

Metrica binaria per singola immagine: 1 se l'oggetto più importante secondo la ground truth è nei primi k posti della predizione. La media sul dataset diventa una percentuale.

Perché Top-1 conta più delle altre in questo progetto. Lo scenario applicativo è l'assistenza visiva: un sistema che descrive la scena a chi non la vede bene. Sbagliare l'ordine fra il terzo e il quarto oggetto è irrilevante; sbagliare *quale sia* l'oggetto principale compromette l'intera descrizione.

11 src/ranking/objects.py

Il passaggio finale: dalle saliency map a una classifica di oggetti.

11.1 L'architettura del confronto

Percorso	Produce
saliency map + bounding box	ranking predetto
click annotati + bounding box	ranking di riferimento

I due percorsi condividono gli stessi box e differiscono solo per la fonte dell'importanza. Il confronto avviene con Spearman, Kendall e Top-1.

11.2 crop_box

```
x1 = int(np.clip(x1, 0, width))
x2 = int(np.clip(x2, 0, width))
y1 = int(np.clip(y1, 0, height))
y2 = int(np.clip(y2, 0, height))

if x2 <= x1 or y2 <= y1:
    return np.empty((0, 0), dtype=saliency_map.dtype)

return saliency_map[y1:y2, x1:x2]
```

Il `np.clip` gestisce i box che sfiorano i bordi: i detector li producono regolarmente quando un oggetto è parzialmente fuori inquadratura. Ricondurli dentro è più utile che sollevare un'eccezione a metà valutazione.

Il controllo successivo intercetta i box degeneri e quelli interamente fuori: dopo il clip diventerebbero intervalli vuoti, e `.mean()` su un array vuoto emette un warning restituendo `nan`.

11.3 score_box — le tre strategie

Strategia	Domanda a cui risponde	Dipendenza dall'area
<code>mean</code>	Quanto è saliente in media?	nessuna
<code>max</code>	Contiene il punto più saliente?	nessuna, ma fragile
<code>sum</code>	Quanta attenzione totale raccoglie?	forte

Perché implementarle tutte e tre. Non sono alternative fra cui scegliere a priori: il loro confronto è una delle analisi previste dal progetto. `sum` è inclusa proprio per rendere *misurabile* il bias dell'area, non perché sia sbagliata in assoluto — se l'obiettivo fosse “quale oggetto occupa più attenzione complessiva”, sarebbe la scelta corretta.

Il risultato non è banale: un modello ottimo su tutte le metriche intrinseche può fallire il ranking finale se si sceglie la strategia sbagliata. La qualità della saliency map non basta: conta anche come la si traduce in punteggi per oggetto.

Il default è `mean`: indipendente dall'area e meno fragile di `max`, che dipende da un solo pixel e premia i box grandi per pura probabilità di contenere un picco isolato.

11.4 rank_by_saliency

```
scores = np.array([score_box(saliency_map, b, strategy) for b in boxes])
order = np.argsort(scores)[::-1]

return [
    {"index": int(i), "box": tuple(boxes[i]),
     "score": float(scores[i]), "rank": position + 1}
    for position, i in enumerate(order)
]
```

Il campo `index` è essenziale. La lista è ordinata per punteggio, quindi la posizione nella lista *non* corrisponde più al box originale. Senza `index` sarebbe impossibile allineare i due ranking per il confronto: si finirebbe per confrontare oggetti diversi credendo di confrontare gli stessi.

11.5 rank_by_clicks

```
for click in clicks:
    containing = [
        i for i, (x1, y1, x2, y2) in enumerate(boxes)
        if x1 <= click["x"] < x2 and y1 <= click["y"] < y2
    ]
    if not containing:
        continue

    smallest = min(containing,
                   key=lambda i: (boxes[i][2] - boxes[i][0]) * (boxes[i][3] - boxes[i][1]))
    scores[smallest] += 1.0 / click.get("order", 1)
```

Due decisioni:

Click nello sfondo: scartati

Un click che non cade in nessun box indica qualcosa che il detector non ha rilevato. Non c'è un oggetto a cui attribuirlo.

Box sovrapposti: vince il più piccolo

I box si sovrappongono regolarmente (una persona e il suo volto, un tavolo e gli oggetti sopra). Assegnare al box più piccolo assume che l'annotatore intendesse l'elemento più specifico.

La seconda è un'**euristica**, non una certezza: in alcuni casi l'intenzione poteva essere l'oggetto contenitore. Va dichiarata fra i limiti del metodo nel README.

Il punteggio $1/\text{order}$ è lo stesso usato nella ground truth, per coerenza: primo click 1.0, secondo 0.5, terzo 0.33. La caduta è ripida di proposito — la prima cosa notata è qualitativamente diversa dalla quinta.

11.6 aligned_scores

```
predicted = np.zeros(n_boxes, dtype=np.float64)
reference = np.zeros(n_boxes, dtype=np.float64)

for entry in predicted_ranking:
    predicted[entry["index"]] = entry["score"]
for entry in reference_ranking:
    reference[entry["index"]] = entry["score"]

return predicted, reference
```

Il bug che questa funzione previene. Passando direttamente i punteggi ordinati alle metriche, si confronterebbe “il primo classificato secondo il modello” con “il primo secondo la ground truth” — ottenendo correlazione perfetta *sempre*, indipendentemente da quali oggetti siano.

Un risultato troppo bello per essere vero, e senza alcun errore visibile.

12 src/registry.py

```
_REGISTRY = {
    "Center Prior": (center_prior_saliency, False),
    "Spectral Residual": (spectral_residual_saliency, True),
    "Itti-Koch": (itti_koch_saliency, True),
}

AVAILABLE_MODELS = list(_REGISTRY.keys())

def compute_saliency(model_name, image_bgr):
    if model_name not in _REGISTRY:
        raise KeyError(
            f"Modello sconosciuto: '{model_name}'. "
            f"Disponibili: {'', ' '.join(AVAILABLE_MODELS)}"
        )

    model_fn, needs_image = _REGISTRY[model_name]
    if needs_image:
        return model_fn(image_bgr)
    return model_fn(image_bgr.shape[:2])
```

Un dizionario che mappa il nome leggibile a una coppia (funzione, vuole_immagine_intera). Il flag booleano cattura l'unica differenza fra le firme: il center prior vuole la *forma*, gli altri l'*immagine*.

Il prefisso `_` in `_REGISTRY` segnala che è un dettaglio interno: chi usa il modulo deve passare da `AVAILABLE_MODELS` e `compute_saliency`.

Il messaggio d'errore elenca i modelli disponibili invece di limitarsi a segnalare il fallimento.

Il beneficio concreto. Aggiungere un modello richiede di modificare *soltanto* questo file: una riga nel dizionario. L'app, i test e lo script di valutazione lo vedranno comparire automaticamente, perché iterano su `AVAILABLE_MODELS`.

13 src/visualization.py

13.1 saliency_to_heatmap

```
saliency_uint8 = (np.clip(saliency_map, 0.0, 1.0) * 255).astype(np.uint8)
heatmap_bgr = cv2.applyColorMap(saliency_uint8, cv2.COLORMAP_JET)
return cv2.cvtColor(heatmap_bgr, cv2.COLOR_BGR2RGB)
```

`cv2.applyColorMap` lavora solo su `uint8`, quindi la mappa $[0, 1]$ va riscalata a $[0, 255]$. Il `np.clip` è una difesa: se un modello restituisse valori leggermente fuori range, il cast produrrebbe wrap-around (255.5 diventa 0) con artefatti vistosi.

`COLORMAP_JET` è lo standard de facto nella letteratura sulla saliency: blu = basso, rosso = alto. La conversione finale a RGB serve perché OpenCV produce BGR mentre Gradio si aspetta RGB.

13.2 overlay_saliency

```
return cv2.addWeighted(image_rgb, 1.0 - alpha, heatmap_rgb, alpha, 0.0)
```

`cv2.addWeighted(src1, a, src2, b, gamma)` calcola `src1*a + src2*b + gamma`. Con `a = 1-alpha` e `b = alpha` si ottiene una miscelazione lineare: `alpha=0` mostra solo l'immagine, `alpha=1` solo la heatmap.

13.3 mark_peak

```
annotated = image_rgb.copy() # cv2.circle disegna in-place
py, px = np.unravel_index(np.argmax(saliency_map), saliency_map.shape)

cv2.circle(annotated, (int(px), int(py)), radius + 2, (0, 0, 0), thickness + 2)
cv2.circle(annotated, (int(px), int(py)), radius, (255, 255, 255), thickness)
```

Il `.copy()` è essenziale: senza, `cv2.circle` modificherebbe l'immagine del chiamante.

Vengono disegnati **due** cerchi concentrici, uno nero più spesso sotto e uno bianco più sottile sopra: così il marcatore resta visibile sia su sfondi chiari sia su sfondi scuri.

`np.argmax` restituisce l'indice nell'array *appiattito*; `np.unravel_index` lo riconverte in (riga, colonna), cioè (y, x). Ma `cv2.circle` vuole il centro come (x, y): l'ordine va invertito esplicitamente.

14 app.py

14.1 Principio architetturale

Il file contiene **solo** l'interfaccia. Il calcolo vive in `src/registry.py`, la visualizzazione in `src/visualization.py`.

14.2 prepare_image

```
MAX_SIDE = 800

def prepare_image(image_rgb):
    height, width = image_rgb.shape[:2]
    scale = MAX_SIDE / max(height, width)

    if scale < 1.0:
        image_rgb = cv2.resize(image_rgb,
                                (int(width * scale), int(height * scale)),
                                interpolation=cv2.INTER_AREA)

    return image_rgb, cv2.cvtColor(image_rgb, cv2.COLOR_RGB2BGR)
```

Due responsabilità:

Ridimensionamento condizionale

`scale` si calcola sul lato *maggiore*, così il rapporto d'aspetto è preservato. La condizione `scale < 1.0` fa sì che le immagini piccole non vengano *ingrandite* — interpolare non aggiunge informazione.

Conversione RGB → BGR

Gradio consegna immagini in RGB, i modelli lavorano in BGR. La conversione avviene in un punto unico, all'ingresso.

Il ridimensionamento è necessario per motivi pratici: Itti-Koch costruisce sette livelli di piramide su tre canali più quattro orientazioni. Su una foto da smartphone il calcolo diventa lento senza alcun beneficio, dato che la saliency è un fenomeno a bassa frequenza spaziale.

14.3 process

```
def process(image_rgb, model_name, alpha, show_peak):
    if image_rgb is None:
        return None, None, "Carica un'immagine per iniziare."

    image_rgb, image_bgr = prepare_image(image_rgb)
    saliency = compute_saliency(model_name, image_bgr)

    overlay = overlay_saliency(image_rgb, saliency, alpha=alpha)
    heatmap = saliency_to_heatmap(saliency)
    ...
```

Il controllo iniziale gestisce lo stato transitorio dell'interfaccia: all'apertura non c'è ancora un'immagine. Senza di esso l'app solleverebbe un'eccezione visibile nel browser.

14.4 Il selettore a radio button

```
model_selector = gr.Radio(
    choices=AVAILABLE_MODELS,
    value=AVAILABLE_MODELS[0],
    label="Modello",
)
```

`gr.Radio` permette di selezionare **un solo** modello alla volta — a differenza di `gr.CheckboxGroup`, che ne permetterebbe più d'uno. Le scelte vengono direttamente da `AVAILABLE_MODELS`: aggiungendo un modello al registro, compare qui senza toccare l'app.

14.5 Collegamento degli eventi

```
inputs = [image_input, model_selector, alpha_slider, peak_checkbox]
outputs = [overlay_output, heatmap_output, info_output]

run_button.click(process, inputs=inputs, outputs=outputs)
image_input.change(process, inputs=inputs, outputs=outputs)
model_selector.change(process, inputs=inputs, outputs=outputs)
alpha_slider.change(process, inputs=inputs, outputs=outputs)
peak_checkbox.change(process, inputs=inputs, outputs=outputs)
```

Ogni collegamento dichiara: *quando accade questo evento, chiama `process` passando i valori di `inputs`, e usa il risultato per aggiornare `outputs`.*

L'ordine di `inputs` deve corrispondere **esattamente** all'ordine dei parametri di `process`. È posizionale, non per nome: invertire due voci non produce un errore, solo comportamenti sbagliati.

Oltre al pulsante esplicito, ogni controllo ricalcola automaticamente al cambiamento: cambiando il radio button il risultato si aggiorna subito.

15 annotate.py

15.1 AnnotationSession

Classe che incapsula lo stato della sessione: quale immagine, quali click, quale annotatore. Separata dall'interfaccia per lo stesso motivo dell'app — così è testabile senza browser.

15.1.1 start — ripresa automatica

```
self.store = AnnotationStore(annotator, self.annotations_dir)
done = self.store.annotated_images()

self.index = 0
while self.index < len(self.image_names) and self.image_names[self.index] in done:
    self.index += 1
```

Il ciclo salta le immagini già annotate *da quell'annotatore*. Chi riapre il browser il giorno dopo riprende esattamente da dove aveva lasciato, senza rifare il lavoro.

15.1.2 save_and_advance

```
if len(self.clicks) < MIN_CLICKS:
    return False, f"Servono almeno {MIN_CLICKS} click (ne hai {len(self.clicks)})."

height, width = self.current_image.shape[:2]
self.store.save_annotation(self.current_name, (height, width), self.clicks)

self.index += 1
self.clicks = []
self._load_current()
return True, "Salvato."
```

Restituisce una coppia (`successo, messaggio`) invece di sollevare eccezioni: in un'interfaccia, un vincolo non rispettato è una condizione normale da comunicare all'utente, non un errore di programmazione.

Notare l'ordine: si salva *prima* di avanzare. Se il salvataggio fallisse, l'indice non avanzerebbe e il lavoro non andrebbe perso.

15.2 draw_clicks

```
for index, (x, y) in enumerate(clicks, start=1):
    cv2.circle(annotated, (x, y), 16, (0, 0, 0), -1)
    cv2.circle(annotated, (x, y), 14, (255, 80, 40), -1)

    text = str(index)
    (tw, th), _ = cv2.getTextSize(text, cv2.FONT_HERSHEY_SIMPLEX, 0.6, 2)
    cv2.putText(annotated, text, (x - tw // 2, y + th // 2), ...)
```

Il **numero** sopra ogni marcatore rende visibile l'*ordine*, che è metà del dato raccolto: senza indicazione visiva l'annotatore perde il conto di cosa ha già cliccato.

`cv2.getTextSize` misura il testo prima di disegnarlo, così da centrarlo esattamente sul marcatore: `x - tw // 2` sposta a sinistra di metà larghezza.

15.3 Il gestore dei click

```
def on_click(event: gr.SelectData):
    if session.current_image is None:
        return None, session.status()
    x, y = event.index[0], event.index[1]
    ...
```

`gr.SelectData` è l'oggetto evento di Gradio per il click su un'immagine. `event.index` contiene `[x, y]` — colonna, riga: l'ordine opposto a quello di numpy.

16 scripts/run_evaluation.py

Entry point che collega tutti i componenti.

16.1 Il controllo di coerenza dimensionale

```
if (height, width) != tuple(record["image_size"]):
    print(f"  ATTENZIONE: {image_name} e' {(height, width)} ma le annotazioni "
          f"sono per {tuple(record['image_size'])}. Saltata.")
    continue
```

Perché è essenziale. Le coordinate dei click sono relative alla risoluzione mostrata all'annotatore. Se l'immagine su disco venisse ridimensionata dopo l'annotazione, i click punterebbero a posizioni diverse — e la valutazione produrrebbe numeri plausibili ma sbagliati, senza alcun errore visibile. È il motivo per cui `image_size` viene salvato con i click.

16.2 Gestione dei NaN nelle medie

```
values = np.array([r[metric] for r in model_rows], dtype=np.float64)
valid = values[~np.isnan(values)]

if valid.size == 0:
    entry[f"{metric}_mean"] = float("nan")
else:
    entry[f"{metric}_mean"] = round(float(valid.mean()), 4)
```

I `nan` restituiti dalle metriche non definite vengono esclusi: senza il filtro, un singolo `nan` renderebbe `nan` l'intera colonna.

16.3 Le due tabelle prodotte

File	Contenuto
<code>metrics_per_image.csv</code>	Una riga per coppia (immagine, modello). Permette di individuare i casi in cui un modello fallisce.
<code>metrics_summary.csv</code>	Media e deviazione standard per modello. È la tabella del report finale.

La deviazione standard non è decorativa: un modello con media alta ma varianza altissima è meno affidabile di uno leggermente peggiore ma costante.

17 tests/test_all.py

56 controlli. La strategia: ogni verifica usa casi in cui la risposta corretta è nota *per costruzione o per ragionamento matematico*.

Perché non basta confrontarsi con un'altra libreria. Un test che verifica solo l'accordo con `scipy` non dice se si è capita la metrica: se si sbaglia una convenzione (per esempio l'ordine degli argomenti di `KL`) il confronto potrebbe comunque passare. I casi con verità nota verificano il *significato*, non solo la coincidenza numerica.

17.1 Esempi di verità note

Caso	Valore atteso
Mappa confrontata con se stessa	$CC = 1$, $SIM = 1$, $KL = 0$
Mappa e il suo opposto	$CC = -1$
Center prior 240×320	picco esattamente in $(160, 120)$
Ordinamento invertito	$\rho = -1$
Una coppia invertita su sei	$\tau = 2/3$
Una fissazione sul minimo	$AUC = 0.5$ (limite inferiore)

17.2 Il test che documenta un limite

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
square = gray[y0:y1, x0:x1].mean()
mask = np.ones(gray.shape, bool)
mask[y0:y1, x0:x1] = False

check(abs(square - gray[mask].mean()) < 10,
      f"quadrato ({square:.0f}) e sfondo ({gray[mask].mean():.0f}) "
      f"isoluminanti: SR non puo' vederlo, Itti-Koch si'")
```

L'immagine di test è un quadrato **rosso** su sfondo **verde**. In scala di grigi il quadrato ha luminanza media **94**, lo sfondo **98**: sono praticamente *isoluminanti*.

Un risultato che vale come argomento all'orale. Spectral Residual converte in scala di grigi come primo passo, quindi per lui quel quadrato *non esiste*: si distingue solo per tinta. Itti-Koch, avendo canali di opponenza cromatica espliciti, lo trova senza difficoltà.

Non è un bug ma una **limitazione strutturale**, e mostra che i modelli non sono ordinabili su una scala unica: falliscono in modi diversi, su input diversi. Genera anche una previsione verificabile sul dataset reale — Spectral Residual dovrebbe andare peggio su scene dove l'oggetto interessante si distingue per colore ma non per luminosità.

17.3 Isolamento tramite cartelle temporanee

```
tmp = tempfile.mkdtemp()
try:
    ...
finally:
    shutil.rmtree(tmp)
```

Ogni test che tocca il disco lavora in una cartella temporanea, ripulita nel `finally` (quindi anche se il test fallisce). I test non toccano mai i dati reali del progetto.

18 Riepilogo

Modello	Usa il contenuto?	Punto debole
Center prior	no	ignora tutto
Spectral Residual	sì	cieco al colore puro
Itti-Koch	sì	nessuna semantica

Il limite comune a tutti e tre: **nessuno capisce cosa sia un oggetto**. Rispondono a contrasto e statistica di basso livello. Un volto umano e una macchia colorata della stessa dimensione e contrasto ricevono lo stesso trattamento — mentre per un osservatore umano il volto vince sempre.

È esattamente il divario che i modelli deep learning, addestrati su fissazioni umane reali, dovrebbero colmare: l'estensione naturale del progetto.

18.1 Domande di autoverifica

Provare a rispondere a mente, senza rileggere:

1. Perché `center_prior_saliency` accetta `image_shape` e non l'immagine?
2. Cosa succederebbe se Spectral Residual scartasse la fase e ne usasse una casuale?
3. Perché in Itti-Koch si divide per l'intensità nel calcolo dei canali di colore?
4. L'operatore $N(\cdot)$ amplifica o sopprime una mappa con molti picchi simili? Perché?
5. Perché la ground truth ha due rappresentazioni invece di una?
6. Come si giustifica il valore $\sigma = 5\%$ della larghezza?
7. Perché la strategia `sum` favorisce sistematicamente gli oggetti grandi?
8. Cosa succede se si passano alle metriche i due ranking già ordinati per punteggio?
9. Con una sola fissazione, qual è il valore minimo possibile dell'AUC-Judd? Perché?
10. Perché aggregare i click di tutti gli annotatori è diverso dal mediare le loro heatmap?